

# CallHub — Module B

---

## First, What is the Big Picture?

Imagine your college has 5000 students, professors and staff. You need to find someone's phone number urgently. Right now you have to ask around or search through Excel sheets. CallHub solves this — it's a **digital phone directory** where anyone can search for anyone and get their contact instantly.

Module B is about building the **actual working system** — the website you open in a browser, the backend that handles all the logic, and making the database fast.

---

## What Technologies Did We Use and Why?

**Python Flask** — This is our backend framework. Think of it like the brain of the system. When you click a button on the website, Flask handles what happens next.

**MySQL** — This is our database. All 5000+ member details, search history, login records, audit logs — everything is stored here. We designed this in Assignment 1.

**HTML/CSS/JavaScript** — This is the frontend — what you actually see and click in the browser.

**JWT (JSON Web Token)** — This is how we handle login sessions. When you login, you get a special token (like a temporary ID card). Every time you do something on the website, this token is sent to prove who you are.

---

## SubTask 1 — Setting Up the Database

**What is it?** Before building the website we needed to set up the database locally on our computer.

**What did we do?** We used the same 14 tables from Assignment 1. We opened MySQL Workbench and ran our SQL files to create all the tables and insert sample data.

**One important thing — no data duplication:** The assignment said don't duplicate login credentials. So we didn't create a separate users table with passwords. Instead we use:

- **Email** = `iit_email` column in Member table
- **Password** = `primary_phone` column in Member table

This is smart because the data already exists — no need to store it twice.

**We also added one new table — `Edit_Request`:** This stores requests from regular users who want to change their details. It has columns for what they want to change, when they requested it, and whether admin accepted or rejected it.

**We seeded 5000+ members:** We wrote a MySQL stored procedure (a small SQL program) that automatically inserted 5000 dummy members. This was needed to make the benchmarking meaningful — with only 20 members you can't see performance differences properly.

---

## SubTask 2 — Building the Website and APIs

This is the biggest part. We built two things — the backend APIs and the frontend website.

### What is an API?

API stands for Application Programming Interface. Think of it like a waiter in a restaurant. You (the website) tell the waiter (API) what you want. The waiter goes to the kitchen (database), gets the food (data), and brings it back to you.

Every button click on our website calls an API behind the scenes.

### Our Backend Files

**`db.py`** — This file contains just one function that connects to MySQL. Every other file imports this to talk to the database. Think of it as the phone line to the database.

**`auth.py`** — This handles everything related to login:

- When you login, it checks your email and phone in the database
- If correct, it generates a JWT token using a secret key
- The token contains your `member_id`, your role, and an expiry time
- It also records your login in the `Login_History` table
- When you logout, it records the logout time

**`rbac.py`** — RBAC stands for Role Based Access Control. This file has two special Python decorators:

- `@login_required` — checks if you have a valid token
- `@admin_required` — checks if you have a valid token AND your role is admin

A decorator in Python is like a security guard placed at the entrance of a function. Before the function runs, the decorator checks your credentials.

## How the System Decides Admin vs Regular User

It's based on the **Role** assigned to the member in the database.

---

# The Flow

**Step 1 — Login:** When you login, the backend runs this query:

```
sql
```

```
SELECT m.member_id, m.member_name, r.role_name
```

```
FROM Member m
```

```
JOIN Member_Role mr ON m.member_id = mr.member_id
```

```
JOIN Role r ON mr.role_id = r.role_id
```

```
WHERE m.iit_email = 'rohit@org.in'
```

```
AND mr.is_primary = TRUE
```

This fetches the member's **role\_name** from the database.

---

**Step 2 — Token Generation:** The role is stored INSIDE the JWT token:

```
python
```

```
payload = {
```

```
    "member_id": 9,
```

```
    "role": "Director", ← stored here
```

```
    "exp": expiry_time
```

```
}
```

---

**Step 3 — Every API Call:** When you call any API, the decorator reads the token and checks the role:

```
python
```

```
ADMIN_ROLES = ["Director", "HOD", "Dean", "Admin Staff", "HR Manager"]
```

```
if payload["role"] not in ADMIN_ROLES:
```

```
    return 403 Forbidden ← blocked
```

---

## So Simply Put

| Role in Database | Access Level |
|------------------|--------------|
| Director         | Admin        |
| HOD              | Admin        |
| Dean             | Admin        |
| Admin Staff      | Admin        |
| HR Manager       | Admin        |
| Student          | Regular User |
| Professor        | Regular User |
| Anyone else      | Regular User |

---

## Who Assigns the Role?

The role is assigned when a member is created — either through:

- `Sample_Data.sql` — we manually assigned roles to the 20 members
  - Add Member form — admin selects the role when adding a new member
- 

## Can Someone Change Their Own Role?

**No!** Only an admin can assign roles. A regular user has no API access to change their own role. Even if they try to modify the JWT token, it will fail because the token is signed with a secret key — any modification makes it invalid.

**routes/members.py** — This handles all member operations:

- GET /members — fetch all members with pagination
- GET /members/id — fetch one member
- POST /members — add new member (admin only)
- PUT /members/id — edit member (admin only)
- DELETE /members/id — deactivate member (admin only)
- POST /members/bulk — upload CSV to add many members at once

**routes/departments.py** — Same as members but for departments:

- GET, POST, PUT, DELETE for departments
- All changes require admin access
- Cannot delete a department that has active members

**routes/analytics.py** — This handles:

- Search — when you type in search box, this API runs the query and logs it to Search\_Log table
- Interact — when you click Call or Email, this logs it to Directory\_Interaction\_Log
- Analytics — shows top searches, most contacted departments, most viewed profiles
- Login History — shows all login sessions

**routes/requests.py** — This handles the edit request system:

- POST /requests — regular user submits a change request
- GET /requests — admin sees all pending requests
- POST /requests/id/accept — admin accepts, changes are applied automatically
- POST /requests/id/reject — admin rejects the request

## Our Frontend Pages

**login.html** — The first page you see. Has email and password fields. When you click Sign In, JavaScript sends the credentials to the /login API. If successful, the JWT token is saved in browser localStorage (temporary browser storage).

**dashboard.html** — This page is completely different depending on your role:

- **Admin sees:** Stats cards (total members, departments, searches, audits), recent activity table, quick action buttons
- **Regular user sees:** Just a big search box with live dropdown and recent contacts

**members.html** — Shows all members in a table with:

- Pagination (20 per page with Prev/Next buttons)

- Search bar with real time filtering
- Department and role filter dropdowns
- For admin: Add, Edit, Delete buttons
- For regular user: Only View button

**portfolio.html** — Shows one member's full profile:

- Name, email, phone, department, role, join date, campus status
- If viewing someone else: Call button (opens phone dialer) and Email button (opens mail app)
- If viewing own profile: Edit request form to submit change requests

**departments.html** — Shows all 20 departments with their building and hours. Admins can add, edit or delete departments.

**analytics.html** — Admin only page showing:

- Top 5 most searched keywords
- Most contacted departments
- Most viewed profiles
- Recent audit log entries
- Full login history

**requests.html** — Admin only page showing all edit requests with Accept and Reject buttons. Can filter by Pending, Accepted or Rejected.

## How a Typical Request Works (Step by Step)

Let's say Amit (regular user) searches for "Priya":

1. Amit types "Priya" in the search box
2. JavaScript waits 250ms (to avoid too many requests while typing)
3. JavaScript sends a POST request to /search with the keyword and Amit's JWT token
4. Flask receives the request
5. @login\_required decorator checks the JWT token — valid 
6. The search query runs on MySQL — finds all members with "Priya" in their name
7. The search is recorded in Search\_Log table
8. Results are sent back as JSON
9. JavaScript shows the results as a dropdown
10. Amit clicks on Priya's name
11. Browser goes to /portfolio/2
12. A VIEW\_PROFILE interaction is logged to Directory\_Interaction\_Log

---

## SubTask 3 — Role Based Access Control (RBAC)

**What is RBAC?** It's a security system where what you can do depends on your role. Just like in a real college — a student can't access the admin office and change their own grade. They have to submit a request.

## Our two roles:

**Admin roles:** Director, HOD, Dean, Admin Staff, HR Manager These people have full control over the system.

**Regular user roles:** Student, Professor, Assistant Professor, etc. These people can only search and view.

## How it works technically:

When you login, your role is stored inside your JWT token. When you call any API, the decorator reads your token, extracts your role, and decides whether to allow or block the request.

For example when Amit (Student) tries to add a member:

1. He sends POST request to `/members`
2. `@admin_required` decorator runs
3. It reads his token — role is "Student"
4. "Student" is not in `ADMIN_ROLES` list
5. It writes an `UNAUTHORIZED` entry to `audit.log`
6. Returns 403 Forbidden — request blocked

**The edit request system:** Since regular users can't edit their own details directly, we built a request system. They fill a form on their profile page saying what they want to change. Admin sees it in the Requests page and can accept or reject. If accepted, the changes are automatically applied to the database.

---

## SubTask 4 — SQL Indexing

### What is an index?

Think of a dictionary. If you want to find the word "elephant" you don't read every page from page 1. You go to the E section directly. That's exactly what a database index does.

Without an index, MySQL reads every single row in the table (called a full table scan). With 5000 members, that means checking 5000 rows every time someone searches. With an index, MySQL goes directly to the relevant rows.

### Which columns did we index and why?

We looked at every SQL query in our APIs and found which columns appear in WHERE (filter conditions), JOIN (connecting tables) and ORDER BY (sorting) clauses.

**member\_name on Member table:** Used in search queries like `WHERE member_name LIKE '%amit%'`. Without index, MySQL scans all 5000 rows. With index, it can narrow down faster.

**iit\_email on Member table:** Used in login — `WHERE iit_email = 'amit@org.in'`. This is used every time someone logs in so it must be fast.

**department\_id on Member table:** Used in JOINS with Department table and in filter queries. Very frequently used.

**member\_id on Member\_Role table:** Every member query JOINS with Member\_Role to get the role name. This JOIN uses member\_id so indexing it speeds up all member queries.

**search\_time on Search\_Log:** Used in analytics queries that ORDER BY search\_time to get recent searches.

**actor\_member\_id and target\_member\_id on Directory\_Interaction\_Log:** Used in analytics queries to find who interacted with whom.

**member\_id on Login\_History:** Used in login history queries.

**performed\_by\_member\_id on Audit\_Log:** Used in audit log queries to find who performed which action.

---

## SubTask 5 — Performance Benchmarking

**What is benchmarking?** It's measuring and comparing performance. Like timing yourself running 100 meters before and after training.

**How did we benchmark?**

**Method 1 — Python script (benchmark.py):**

Step 1 — Remove all indexes so MySQL has no shortcuts Step 2 — Run each query 10 times and record each time using `time.perf_counter()` Step 3 — Calculate the average of 10 runs (averaging removes random fluctuations) Step 4 — Add all indexes back Step 5 — Run the same queries 10 times again Step 6 — Compare averages and calculate improvement percentage

We ran 10 times instead of once because computer speed varies slightly each run due to CPU load and memory. Averaging 10 runs gives a more accurate result.

**Method 2 — MySQL SHOW PROFILES:**

This is a built-in MySQL tool that measures exact query execution time.

```
sql
SET profiling = 1; -- turn on profiling
-- run the query
SHOW PROFILES;    -- see the times
```

**Our Results:**

| Query                         | Before Index | After Index | Improvement   |
|-------------------------------|--------------|-------------|---------------|
| Name search (Python)          | 2.6477ms     | 1.5132ms    | 42.85% faster |
| Department filter (Python)    | 0.000664s    | 0.000556s   | 16.3% faster  |
| Name search (MySQL profiling) | 0.01918s     | 0.00773s    | 59.7% faster  |



**Why different improvement percentages?** Because different queries benefit differently from indexes. The department filter improved by 16% because it uses an exact match (`department_id = 1`) which indexes handle perfectly. The name search with `LIKE '%test%'` has a leading wildcard which limits index effectiveness but still showed improvement on larger datasets.

**EXPLAIN command:** We also used `EXPLAIN` which shows MySQL's execution plan — basically how MySQL decided to find the data.

Before index: `type: ALL` — means full table scan, `rows: 4941` After index: `type: ref, key: idx_member_role_mid` — means using index directly

This visually proves MySQL switched from scanning everything to using the index.

---

## Security Logging — Dual Logging

**What is it?** Every admin action is recorded in two places at the same time.

**Place 1 — `logs/audit.log` file:** A simple text file where each line is a JSON entry containing:

- Exact timestamp
- Who performed the action (`member_id`)
- What action (`INSERT`, `UPDATE`, `DELETE`)
- Which table was affected
- Old data (before the change)
- New data (after the change)

**Place 2 — `Audit_Log` MySQL table:** Same information stored in the database so admins can query it, filter it and see it in the Analytics page.

**Why two places?** The file is easy to check quickly and doesn't depend on the database being available. The database table allows complex queries like "show me all deletes in the last week".

**Unauthorized access logging:** When a regular user tries to access an admin endpoint, we don't just block them — we also write an `UNAUTHORIZED_ACCESS` entry to `audit.log`. This makes it easy to detect if someone is trying to bypass security.

---

## Extra Features

**Pagination:** Loading 5000 members at once would crash the browser. So we load 20 at a time. The `/members` API accepts `page` and `limit` parameters. The frontend shows Prev/Next buttons.

**Live Search:** As you type, the search sends requests to the API with a 250ms delay (debouncing). Results appear as a Google-style dropdown. When you click a result it saves to recent contacts in `localStorage`.

**Recent Contacts:** The dashboard shows your recently viewed profiles stored in browser localStorage. You can clear them anytime.

**Call and Email:** Clicking Call uses `window.location.href = 'tel:number'` which opens the phone dialer. Clicking Email uses `window.location.href = 'mailto:email'` which opens the mail app.

---

## Common Questions Professor Might Ask

**Q: Why JWT and not sessions?** JWT is stateless — the server doesn't need to store session data. The token itself contains all the information. This makes it faster and more scalable.

**Q: Why soft delete instead of hard delete?** We set `exit_date` instead of actually deleting the row. This preserves audit history — you can still see who deleted what and when. If you hard delete, all related audit logs become orphaned.

**Q: Why is the LIKE '%test%' improvement small?** Because LIKE with a leading wildcard (%test%) can't use a B-Tree index efficiently. MySQL still has to scan all rows. A trailing wildcard (test%) would use the index perfectly. This is a known MySQL limitation.

**Q: What happens if the JWT token is stolen?** The token expires in 2 hours. After that it becomes invalid automatically. We could improve this by adding a token blacklist on logout but for this assignment the expiry is sufficient.

**Q: How does RBAC work at the API level?** The `@admin_required` decorator runs before the actual function. It reads the Authorization header, decodes the JWT token, checks the role field inside the token, and either allows or blocks the request. The frontend also hides buttons but the real security is at the API level.

**Q: Why did you use Flask instead of Django?** Flask is lightweight and simple — perfect for a project of this size. Django has too much overhead for what we needed.

**Q: What is the difference between Search\_Log and Directory\_Interaction\_Log?** Search\_Log records when someone types a keyword and searches. Directory\_Interaction\_Log records when someone actually views a profile, clicks call or clicks email. Search\_Log tracks intent, Interaction\_Log tracks actual communication actions.

**Q: How does the edit request system work?** Regular user fills a form on their own profile page. The request is saved to `Edit_Request` table with status `PENDING`. Admin sees it on the Requests page. If accepted, the API automatically updates the Member table with the new values and marks the request as `ACCEPTED`. Everything is logged to `audit.log`.

# IMPROVEMENT:

## First time login:

- User gets a temporary password (their phone number)
- After login, system detects it's first login
- Forces them to set a new password
- From next time they use email + new password

## Forgot password:

- User clicks "Forgot Password"
  - System sends OTP to their email
  - User enters OTP
  - User sets new password
- 

## Changes

### Database:

- Add password\_hash column to Member table
- Add is\_first\_login boolean column
- Add otp and otp\_expiry columns

### Backend:

- Update login to check password\_hash
- Add /change-password API
- Add /forgot-password API
- Add /verify-otp API

### Frontend:

- Add change password page
- Add forgot password page